Foundations of Neural Networks & Machine Learning

A Comprehensive Tutorial: From Data Preprocessing to Classifier Interpretability

Astronomy Club

July 17, 2026

Tutorial Roadmap

This tutorial covers the complete pipeline of machine learning classifiers, starting from raw tabular physical measurements to interpreting black-box neural networks:

Module 1: Data Preprocessing & Feature Scaling
- Physical scales, Standardization vs. Normalization, and train-test scaling isolation.

Module 2: Neural Network Architecture & Forward Propagation
- The artificial neuron, hidden layers, matrix mechanics, and activation functions.

Module 3: Multiclass Classification & Loss Functions
- Softmax normalization, Categorical Cross-Entropy, and loss math.

Module 4: Backpropagation & Optimization Math
- Calculus chain rule, logit gradient derivations, and L2 weight decay.

Module 5: Generalization Hyperparameter Search
- Data leakage, Stratification, K-Fold Cross-Validation, and Grid Search.

Module 6: Model Diagnostics & Interpretability

Module 1: Data Preprocessing & Feature Scaling

Why Do We Scale Data?

In astronomical and physical datasets, different features represent distinct physical quantities and vary across vastly different numerical scales:

- **Redshift (z):** Typically a small decimal number, e.g., $z \in [0.01, 0.5]$ in local surveys.
- **Stellar Mass ($\log_{10} M_\star$):** Logarithmic solar mass units, e.g., $\log_{10} M_\star \in [8.0, 12.0]$ (10^8 to $10^{12} M_\odot$).
- **Star Formation Rate (SFR):** Logarithmic units, e.g., $\log_{10} \mathrm{SFR} \in [-5.0, 2.0]$.

The Mathematical Hazard of Raw Features

If we feed unscaled features into gradient-based optimizers (like neural networks):

1. Features with larger absolute scales (like Stellar Mass) will dominate the weight updates.
2. The loss surface will become highly elongated and anisotropic (stretched like a deep valley).
3. Gradient descent will oscillate wildly, requiring an extremely small learning rate to avoid exploding gradients, leading to extremely slow convergence.

Standardization vs. Normalization

There are two primary methods for scaling physical features before feeding them to classifiers:

Normalization (Min-Max Scaling): Rescales features linearly to fit within a fixed range, typically $[0, 1]$:

$$x_{\text{norm}} = \frac{x - x_{\text{min}}}{x_{\text{max}} - x_{\text{min}}}$$

- Drawback: Highly sensitive to outliers. A single extreme outlier (e.g., a hyper-luminous quasar) compresses the normal galaxy population into a tiny, indistinguishable range.

Standardization (Z-Score Scaling): Centers the feature to have a mean of 0 and scales it to have a standard deviation of 1:

$$x_{\text{scaled}} = \frac{x - \mu}{\sigma}$$

where μ is the mean and σ is the standard deviation.

- Advantage: Robust to outliers. Preserves the relative variance of features and ensures zero-centered symmetric input distributions, which aids gradient descent.

Hand-Worked Preprocessing Example

Let's standardize the Stellar Mass ($\log_{10} M_\star$) feature for a sample of $N = 3$ host galaxies:

$$\mathbf{x} = [9.0, 10.0, 11.0]^T \quad \text{(in units of } \log_{10} M_\odot\text{)}$$

1. **Step 1: Compute the sample mean (μ):**

$$\mu = \frac{1}{N}\sum_{i=1}^{N} x_i = \frac{9.0 + 10.0 + 11.0}{3} = 10.0$$

2. **Step 2: Compute the sample standard deviation (σ):**

$$\sigma = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(x_i - \mu)^2} = \sqrt{\frac{(9-10)^2 + (10-10)^2 + (11-10)^2}{3}} = \sqrt{\frac{1+0+1}{3}} \approx$$

3. **Step 3: Standardize each data point ($x_{\text{scaled}} = \frac{x-\mu}{\sigma}$):**
   - $x_1 = 9.0 \implies x_{\text{scaled},1} = \frac{9.0-10.0}{0.8165} \approx -1.225$
   - $x_2 = 10.0 \implies x_{\text{scaled},2} = \frac{10.0-10.0}{0.8165} = 0.0$
   - $x_3 = 11.0 \implies x_{\text{scaled},3} = \frac{11.0-10.0}{0.8165} \approx +1.225$

The resulting standardized vector $\mathbf{x}_{\text{scaled}} \approx [-1.225, 0.0, 1.225]^T$ now has a mean of 0 and variance of 1.

Preprocessing Pitfalls: Train-Test Scaling Isolation

When preparing data for evaluation, we split the dataset into a training set and a testing set. Scaling must be applied with extreme caution.

The Scaling Leakage Trap

A common mistake is scaling the entire dataset before splitting, or fitting the scaler on the test set. If test set samples are used to compute the mean μ and standard deviation σ, statistical information about the test set's distribution leaks into the training phase. This is a form of data leakage.

The Fit-Transform Boundary Rule

To maintain strict validation boundaries:

1. **Fit only on Training Data:** Compute μ_{train} and σ_{train} exclusively using the training set.
2. **Transform both partitions:** Scale the training set using training statistics, and scale the test set using those same training statistics (μ_{train} and σ_{train}).

$$X_{\text{train, scaled}} = \frac{X_{\text{train}} - \mu_{\text{train}}}{\sigma_{\text{train}}}, \quad X_{\text{test, scaled}} = \frac{X_{\text{test}} - \mu_{\text{train}}}{\sigma_{\text{train}}}$$

Module 2: Neural Network Architecture & Forward Propagation

Biological Inspiration & The Artificial Neuron

Neural networks are inspired by the brain's network of biological neurons, which receive electrical signals, integrate them, and fire once a voltage threshold is passed.

The Artificial Neuron (Node)
An artificial neuron is a mathematical node that:

1. Receives a set of numerical inputs: $\mathbf{x} = [x_0, x_1, \ldots, x_d]^T$.
2. Multiplies each input by a corresponding weight parameter: $\mathbf{w} = [w_0, w_1, \ldots, w_d]^T$.
3. Integrates the weighted inputs and adds a bias parameter b:

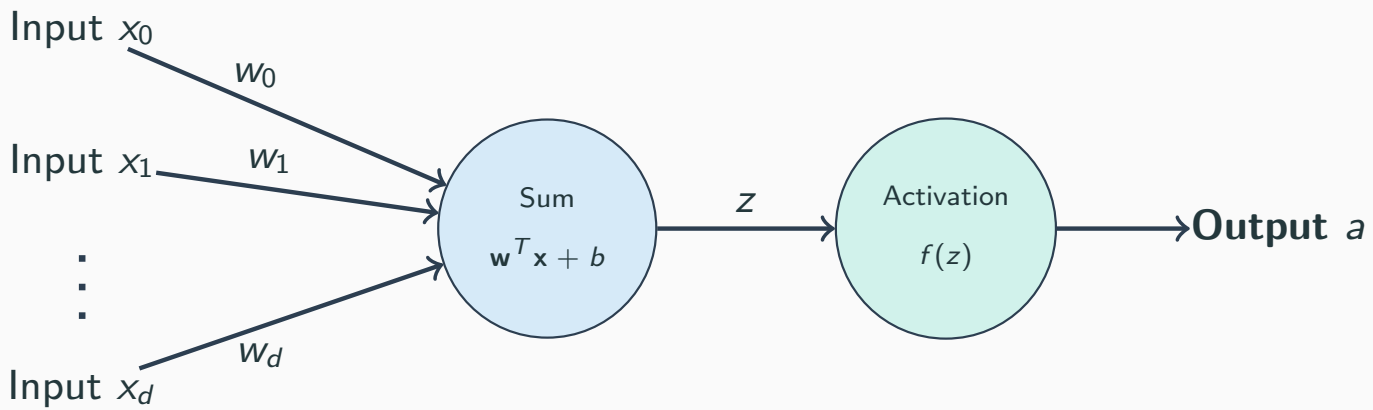
$$z = \sum_{i=1}^{d} w_i x_i + b = \mathbf{w}^T \mathbf{x} + b$$

4. Applies a non-linear activation function $f(z)$ to produce the output activation a:

$$a = f(z) = f(\mathbf{w}^T \mathbf{x} + b)$$

Illustration: The Artificial Neuron Model

Below is a schematic of a single neuron. The inputs are weighted, summed with a bias, and passed through an activation function to generate the output:



Going Deeper: Multi-Layer Perceptrons (MLPs)

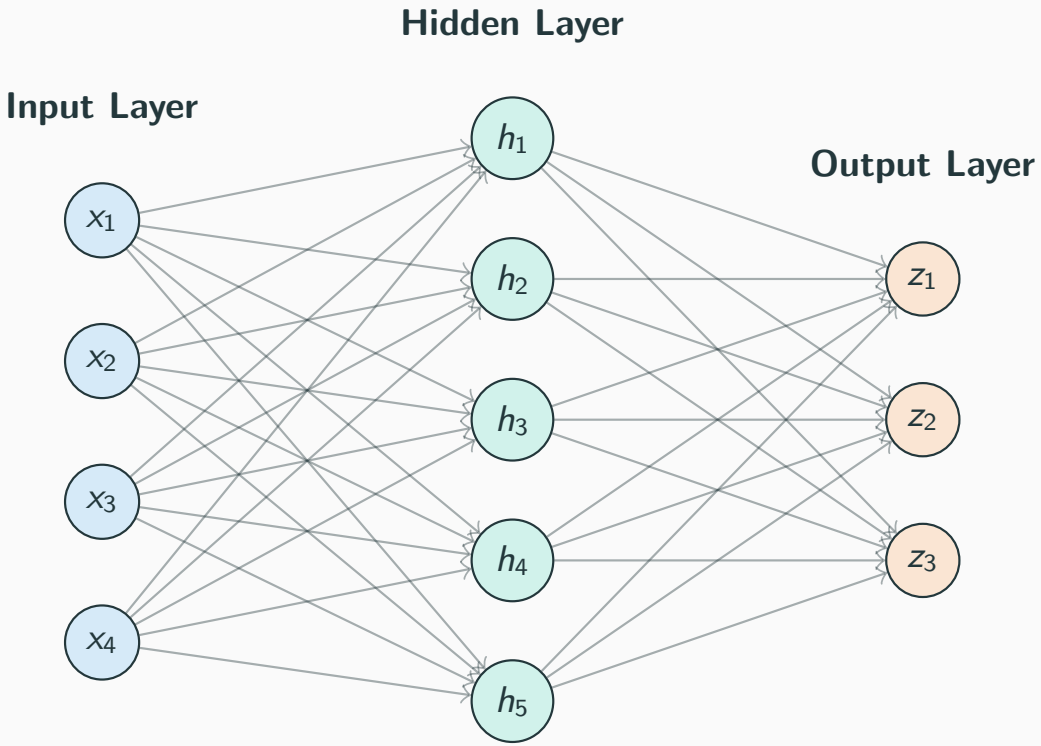
A single neuron can only resolve linear decision boundaries (hyperplanes). To learn complex, curved boundaries, we stack neurons in layers, forming a **Multi-Layer Perceptron (MLP)**:

- **Input Layer ($l = 0$):** Represents the scaled input features. No computations occur here.
- **Hidden Layers ($l = 1 \dots L - 1$):** Intermediate layers that extract higher-level representations. The output of one layer serves as the input to the next.
- **Output Layer ($l = L$):** The final layer, which produces logits or predictions.

An MLP is **fully connected** (or dense), meaning every neuron in layer l receives inputs from every single neuron in the preceding layer $l - 1$.

Illustration: 3-Layer MLP Feedforward Network

Below is a fully connected neural network with 4 inputs ($d = 4$), a hidden layer with 5 neurons ($H = 5$), and 3 output nodes ($C = 3$):



MLP Feedforward Mathematics

To evaluate the network, we compute the activations of each layer sequentially from left to right. For any layer $l \geq 1$:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$$

$$\mathbf{a}^{(l)} = f(\mathbf{z}^{(l)})$$

Where:

- $\mathbf{a}^{(l-1)}$ is the activation vector from the previous layer ($\mathbf{a}^{(0)} = \mathbf{x}_{\text{scaled}}$).
- $\mathbf{W}^{(l)}$ is the weight matrix of layer l, where element $W_{ij}^{(l)}$ represents the weight connecting neuron j in layer $l-1$ to neuron i in layer l.
- $\mathbf{b}^{(l)}$ is the bias vector of layer l.
- $f(\cdot)$ is the activation function, applied element-wise.

This vectorization allows us to leverage highly optimized BLAS (Basic Linear Algebra Subprograms) libraries for matrix operations.

Weight and Bias Dimension Analysis

Understanding matrix dimensions is essential for debugging neural network architectures. Let's analyze a network with:

$$\text{Inputs } d = 4, \quad \text{Hidden Layer Sizes } = (32, 16), \quad \text{Outputs } C = 3$$

Layer 1 (First Hidden Layer):
- Receives input $\mathbf{a}^{(0)} \in \mathbb{R}^4$. Outputs activation $\mathbf{a}^{(1)} \in \mathbb{R}^{32}$.
- Weight matrix $\mathbf{W}^{(1)}$ must map $\mathbb{R}^4 \to \mathbb{R}^{32}$: $\mathbf{W}^{(1)} \in \mathbb{R}^{32\times 4}$.
- Bias vector matches output size: $\mathbf{b}^{(1)} \in \mathbb{R}^{32}$.

Layer 2 (Second Hidden Layer):
- Receives input $\mathbf{a}^{(1)} \in \mathbb{R}^{32}$. Outputs activation $\mathbf{a}^{(2)} \in \mathbb{R}^{16}$.
- Weight matrix $\mathbf{W}^{(2)}$ maps $\mathbb{R}^{32} \to \mathbb{R}^{16}$: $\mathbf{W}^{(2)} \in \mathbb{R}^{16\times 32}$.
- Bias vector matches output size: $\mathbf{b}^{(2)} \in \mathbb{R}^{16}$.

Layer 3 (Output Layer):
- Receives input $\mathbf{a}^{(2)} \in \mathbb{R}^{16}$. Outputs logits $\mathbf{z}^{(3)} \in \mathbb{R}^3$.
- Weight matrix $\mathbf{W}^{(3)}$ maps $\mathbb{R}^{16} \to \mathbb{R}^3$: $\mathbf{W}^{(3)} \in \mathbb{R}^{3\times 16}$.
- Bias vector matches output size: $\mathbf{b}^{(3)} \in \mathbb{R}^3$.

The Necessity of Non-Linear Activations

What happens if we remove the non-linear activation functions $f(z)$ and let $f(z) = z$?

Collapsing Layers Proof
Consider a two-layer linear MLP with inputs $\mathbf{x}$, weights $\mathbf{W}^{(1)}, \mathbf{W}^{(2)}$ and biases $\mathbf{b}^{(1)}, \mathbf{b}^{(2)}$:

$$\mathbf{a}^{(1)} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}$$

$$\mathbf{y}_{\text{pred}} = \mathbf{W}^{(2)}\mathbf{a}^{(1)} + \mathbf{b}^{(2)}$$

Substituting the first equation into the second:

$$\mathbf{y}_{\text{pred}} = \mathbf{W}^{(2)}\left(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}\right) + \mathbf{b}^{(2)}$$

$$\mathbf{y}_{\text{pred}} = \left(\mathbf{W}^{(2)}\mathbf{W}^{(1)}\right)\mathbf{x} + \left(\mathbf{W}^{(2)}\mathbf{b}^{(1)} + \mathbf{b}^{(2)}\right)$$

Let $\mathbf{W}' = \mathbf{W}^{(2)}\mathbf{W}^{(1)}$ and $\mathbf{b}' = \mathbf{W}^{(2)}\mathbf{b}^{(1)} + \mathbf{b}^{(2)}$. Then:

$$\mathbf{y}_{\text{pred}} = \mathbf{W}'\mathbf{x} + \mathbf{b}'$$

This collapses into a single-layer linear regression! Stacking linear layers adds zero representation power. Non-linear activations are required to bend and

Activation Function: ReLU

The **Rectified Linear Unit (ReLU)** is the most widely used activation function in modern deep learning:

$$f(z) = \max(0, z)$$

- **Output Range:** $[0, \infty)$
- **Derivative:**

$$f'(z) = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z < 0 \end{cases}$$

- **Pros:**
  - Extremely fast to calculate (requires only a maximum check).
  - The derivative is 1.0 for all positive inputs, preventing vanishing gradients during backpropagation.
- **Cons:**
  - Not zero-centered.
  - Dying ReLU Problem: If a neuron receives updates that make it negative for all training points, its gradient becomes exactly zero. It will never fire again, becoming a dead weight in the network.

Activation Function: Tanh

The **Hyperbolic Tangent (tanh)** activation function is a smooth, S-shaped curve:

$$f(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

- **Output Range:** $(-1, 1)$
- **Derivative:**

$$f'(z) = 1 - \tanh^2(z)$$

- **Pros:**
  - Outputs are **zero-centered** (mean output is close to 0). This centers the activations for subsequent layers, which speeds up convergence.
- **Cons:**
  - Computationally heavier due to exponentiations.
  - Vanishing Gradient Problem: For large positive or negative inputs, $\tanh(z)$ saturates close to $+1$ or -1. In these regions, the derivative $f'(z) \approx 0$, which stops gradient flow during training.

Module 3: Multi-Class Output & Loss Functions

Multiclass Classification: Logits vs. Probabilities

In multiclass classification (e.g., classifying a galaxy as Class 0: Background, Class 1: CC-SN host, or Class 2: Ia-SN host), the output layer has $C = 3$ neurons.

- The raw real-valued output vector of the final layer is called the **logits** vector:

$$\mathbf{z} = [z_0, z_1, \ldots, z_{C-1}]^T \in \mathbb{R}^C$$

- Logits are unbounded: they can be negative, positive, or zero, and they do not sum to 1.
- In physics and classification, we need a vector of probabilities:

$$\mathbf{P} = [P_0, P_1, \ldots, P_{C-1}]^T$$

where each $P_c \in [0, 1]$ represents the probability that the galaxy belongs to class c, and:

$$\sum_{c=0}^{C-1} P_c = 1.0$$

The Softmax Normalization Function

To convert raw logits $\mathbf{z}$ into a valid probability distribution, we apply the **Softmax function**:

$$P_c = P(y = c \mid \mathbf{x}) = \frac{e^{z_c}}{\sum_{j=0}^{C-1} e^{z_j}}$$

Why do we exponentiate?

1. **Enforces Positivity:** Exponentiating $e^{z_c} > 0$ ensures all probabilities are strictly positive, even if a logit z_c is highly negative.
2. **Enforces Sum-to-One:** Dividing by the sum of all exponentiated logits ensures that $\sum_c P_c = 1.0$ exactly.
3. **Highlights the Max:** Exponentiating accentuates differences: the largest logit receives a disproportionately higher probability, making the model's predictions more decisive.

Hand-Worked Softmax Example

Let's calculate the predicted probabilities for a galaxy that yields raw output logits:

$$\mathbf{z} = [-0.5, 1.5, 0.0]^T$$

representing Class 0 (Background), Class 1 (CC-SN), and Class 2 (Ia-SN).

1. **Step 1: Compute the exponentiated logits (e^{z_c}):**
   - $e^{z_0} = e^{-0.5} \approx 0.6065$
   - $e^{z_1} = e^{1.5} \approx 4.4817$
   - $e^{z_2} = e^{0.0} = 1.0000$
2. **Step 2: Compute the sum of exponentiated logits (denominator):**

$$\sum_{j=0}^{2} e^{z_j} = 0.6065 + 4.4817 + 1.0000 = 6.0882$$

3. **Step 3: Divide each exponentiated logit by the sum:**
   - $P_0 = 0.6065/6.0882 \approx \mathbf{0.100}$ (10.0%)
   - $P_1 = 4.4817/6.0882 \approx \mathbf{0.736}$ (73.6%)
   - $P_2 = 1.0000/6.0882 \approx \mathbf{0.164}$ (16.4%)

The output probability vector is $\mathbf{P} \approx [0.100, 0.736, 0.164]^T$. The network predicts Class 1 with 73.6% confidence.

Loss Functions: Why Not Mean Squared Error (MSE)?

To train a classifier, we need a loss function $\mathcal{L}$ that measures the error between predictions $\mathbf{P}$ and true labels $\mathbf{y}$.

- Why don't we use the standard Mean Squared Error (MSE) from regression?

$$\mathcal{L}_{\text{MSE}} = \frac{1}{C}\sum_{c=0}^{C-1}(P_c - y_c)^2$$

Problems with MSE for Classification

1. **Non-Convexity:** Combining MSE with Softmax or Sigmoid functions creates a highly non-convex loss surface with many local minima.
2. **Vanishing Gradients:** If a model makes a highly confident incorrect prediction (e.g., $P_c \approx 0$ when true label is 1), the gradient of MSE with respect to the input weights becomes extremely small. The network will get stuck and stop learning.

Categorical Cross-Entropy Loss

For multiclass classification, the standard objective function is the **Categorical Cross-Entropy Loss**:

Loss for a single instance i

$$\mathcal{L}_i = -\sum_{c=0}^{C-1} y_c \log P_c$$

where y_c is a one-hot encoded representation of the true label:

$$y_c = \begin{cases} 1 & \text{if class } c \text{ is the true label} \\ 0 & \text{otherwise} \end{cases}$$

- If the true class is k, then $y_k = 1$ and all other $y_{c \neq k} = 0$. The formula simplifies to:

$$\mathcal{L}_i = -\log P_k$$

- **Intuition:** We only care about maximizing the probability of the correct class.
  - If $P_k \to 1.0$, the loss $\mathcal{L}_i \to -\log(1) = 0$.
  - If $P_k \to 0.0$, the loss $\mathcal{L}_i \to -\log(0) = \infty$, heavily penalizing incorrect predictions.

Hand-Worked Cross-Entropy Example

Let's calculate the Cross-Entropy loss for a galaxy with the following parameters:

- **True Label:** Class 1 (CC-SN host). In one-hot encoding:

$$\mathbf{y} = [0, 1, 0]^T$$

- **Predicted Probabilities:** From our Softmax calculations:

$$\mathbf{P} = [0.100, 0.736, 0.164]^T$$

Loss Computation
Applying the Categorical Cross-Entropy formula:

$$\begin{aligned}
\mathcal{L} &= -\sum_{c=0}^{2} y_c \log P_c \\
&= -\left(y_0 \log P_0 + y_1 \log P_1 + y_2 \log P_2\right) \\
&= -\left(0 \cdot \log(0.100) + 1 \cdot \log(0.736) + 0 \cdot \log(0.164)\right) \\
&= -\log(0.736) \\
&\approx -(-0.3065) \approx \mathbf{0.3065}
\end{aligned}$$

If the predicted probability for the true class was lower, e.g., $P_1 = 0.20$, the loss would be $-\log(0.20) \approx 1.609$, which is much higher.

Module 4: Backpropagation & Optimization Math

How Networks Learn: Gradient Descent

A neural network is trained by finding weights $\mathbf{W}$ and biases $\mathbf{b}$ that minimize the average loss across all N training samples:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}_i$$

We update the weight parameters using **Gradient Descent**:

$$\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}}$$

where $\eta > 0$ is the learning rate.

Backpropagation
To compute the partial derivatives $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}}$ for every layer, we use the **Chain Rule of Calculus**, propagating the error gradients backward from the output layer to the input layer.

Softmax Derivative Jacobian (Part 1 - Setup)

Let's derive the derivative of the Softmax output P_k with respect to the input logit z_c. Recall that $P_k = e^{z_k} / \sum_j e^{z_j}$. Let $\Sigma = \sum_j e^{z_j}$. Using the Quotient Rule:

$$\left(\frac{u}{v}\right)' = \frac{u'v - uv'}{v^2}$$

We set $u = e^{z_k}$ and $v = \Sigma$.

- The derivative of the denominator is:

$$\frac{\partial \Sigma}{\partial z_c} = \frac{\partial}{\partial z_c} \sum_j e^{z_j} = e^{z_c}$$

- The derivative of the numerator $u = e^{z_k}$ depends on whether $k = c$:

$$\frac{\partial e^{z_k}}{\partial z_c} = \delta_{kc} e^{z_k}$$

  where Kronecker's delta $\delta_{kc} = 1$ if $k = c$, and 0 otherwise.

Softmax Derivative Jacobian (Part 2 - Calculation)

Substituting these derivatives into the quotient rule:

$$\begin{aligned}
\frac{\partial P_k}{\partial z_c} &= \frac{(\delta_{kc} e^{z_k}) \Sigma - e^{z_k} (e^{z_c})}{\Sigma^2} \\
&= \frac{\delta_{kc} e^{z_k}}{\Sigma} - \frac{e^{z_k} e^{z_c}}{\Sigma^2} \\
&= \delta_{kc} \left(\frac{e^{z_k}}{\Sigma} \right) - \left(\frac{e^{z_k}}{\Sigma} \right) \left(\frac{e^{z_c}}{\Sigma} \right) \\
&= \delta_{kc} P_k - P_k P_c \\
&= P_k (\delta_{kc} - P_c)
\end{aligned}$$

Softmax Gradient Summary

- If $k = c$ (diagonal elements): $\frac{\partial P_c}{\partial z_c} = P_c(1 - P_c)$
- If $k \neq c$ (off-diagonal elements): $\frac{\partial P_k}{\partial z_c} = -P_k P_c$

Cross-Entropy Loss Logit Gradient (Part 1 - Chain Rule)

Now, let's derive the gradient of the single-galaxy Cross-Entropy loss $\mathcal{L}_i = -\sum_k y_k \log P_k$ with respect to the output logits z_c. Applying the Chain Rule:

$$\begin{aligned}\frac{\partial \mathcal{L}_i}{\partial z_c} &= -\sum_{k=0}^{C-1} y_k \frac{\partial \log P_k}{\partial z_c} \\ &= -\sum_{k=0}^{C-1} y_k \frac{1}{P_k} \frac{\partial P_k}{\partial z_c}\end{aligned}$$

Now we substitute our Softmax Jacobian result $\frac{\partial P_k}{\partial z_c} = P_k(\delta_{kc} - P_c)$ into this equation.

Cross-Entropy Loss Logit Gradient (Part 2 - Derivation)

Performing the substitution and simplifying:

$$\begin{aligned}\frac{\partial \mathcal{L}_i}{\partial z_c} &= -\sum_{k=0}^{C-1} y_k \frac{1}{P_k} \left[P_k(\delta_{kc} - P_c)\right] \\ &= -\sum_{k=0}^{C-1} y_k(\delta_{kc} - P_c) \\ &= -\sum_{k=0}^{C-1} y_k \delta_{kc} + \sum_{k=0}^{C-1} y_k P_c \\ &= -y_c + P_c \sum_{k=0}^{C-1} y_k\end{aligned}$$

Since $\mathbf{y}$ is a probability distribution (one-hot encoded), it sums to exactly 1.0 ($\sum_k y_k = 1.0$).

$$\frac{\partial \mathcal{L}_i}{\partial z_c} = P_c - y_c$$

The Gradient Rule

The error gradient at the output logit is simply the linear difference between predicted probability and ground truth label: $P_c - y_c$.

Regularization: Overfitting & L2 Weight Decay

A neural network with many hidden units has high capacity and can easily overfit the training set, memorizing statistical noise rather than physical relationships. To prevent this, we add an **L2 Regularization penalty** (also known as weight decay) to the objective function:

$$\mathcal{L} = \mathcal{L}_0 + \frac{\alpha}{2} \sum_w w^2$$

where $\mathcal{L}_0$ is the standard unregularized cross-entropy loss, and $\alpha > 0$ is the regularization strength.

Gradient Update with Weight Decay
Taking the derivative with respect to a single weight w:

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}_0}{\partial w} + \alpha w$$

The gradient descent update rule becomes:

$$w \leftarrow w - \eta \frac{\partial \mathcal{L}}{\partial w} = w - \eta \left(\frac{\partial \mathcal{L}_0}{\partial w} + \alpha w \right) = (1 - \eta\alpha) w - \eta \frac{\partial \mathcal{L}_0}{\partial w}$$

At each step, the weight is first shrunk by a factor $(1 - \eta\alpha) < 1.0$, penalizing large weights and encouraging simpler decision boundaries.

Module 5: Generalization & Hyperparameter Search

Hyperparameters vs. Model Parameters

When training classifiers, it is important to distinguish between two types of variables:

Model Parameters: Weights ($\mathbf{W}$) and biases ($\mathbf{b}$). These are learned directly from the training data by minimizing the loss function via backpropagation.

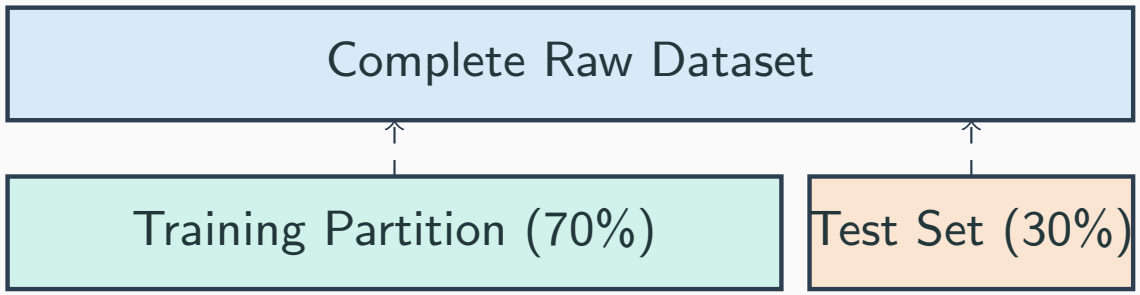
Hyperparameters: External settings configured before training begins. They control the training process and model structure, e.g.:

- Network architecture: number of hidden layers and layer widths.
- Activation functions: ReLU vs. tanh.
- Learning rate η and training epoch count.
- Regularization strength α (weight decay).

Hyperparameters cannot be learned via standard gradient descent. Instead, we must search across a hyperparameter grid to find the optimal configuration.

Validation Strategies: Train-Validation-Test Splits

To evaluate hyperparameters, we partition our raw dataset:



- **Training Partition:** Used to optimize the model weights.
- **Test Set:** Isolated from all training and tuning steps. Used only at the end to evaluate model generalization.
- Stratification: Because classes are imbalanced (150 field, 30 CC-SN, 20 Ia-SN), we must use stratified splitting. This ensures that the class ratios are preserved exactly across both splits, preventing scenarios where a class is entirely missing from a partition.

K-Fold Cross-Validation Mechanics

If we tune hyperparameters on a single train-validation split, we risk overfitting to that specific validation set. To get a robust estimate of generalization, we use **K-Fold Cross-Validation**:

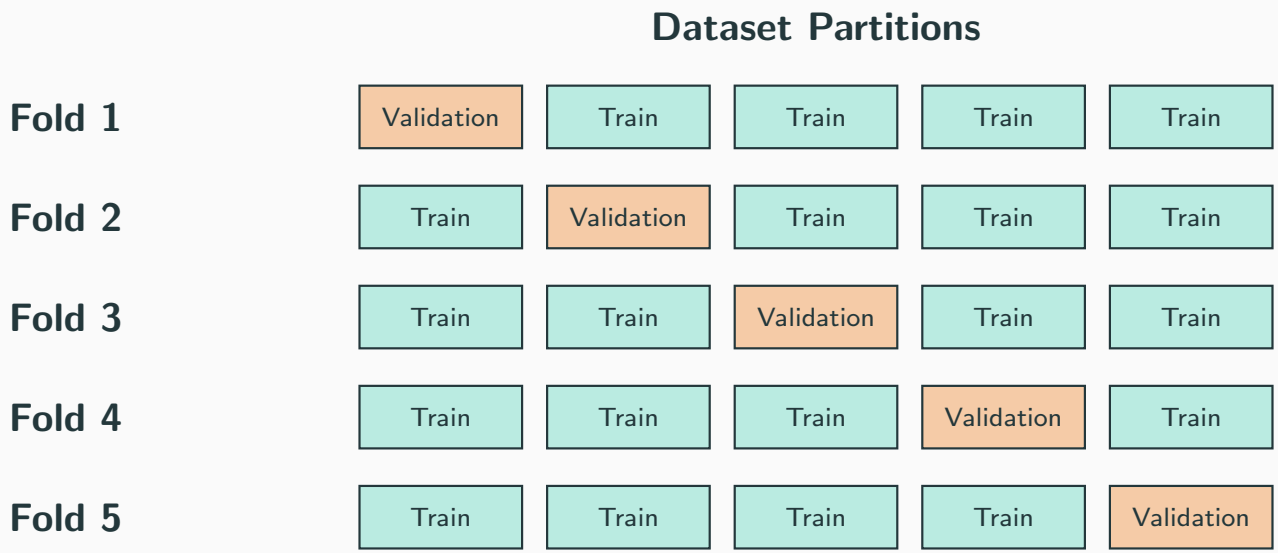
The K-Fold Algorithm

1. Partition the training dataset into K equal-sized, stratified folds.
2. Loop through each fold $k \in \{1 \dots K\}$:
   - Treat fold k as the validation set.
   - Combine the remaining $K - 1$ folds to form the temporary training set.
   - Train the model on the temporary training set.
   - Compute the evaluation metric (e.g., Macro F1) on the validation fold k.
3. Average the validation metrics across all K runs to get a stable score for that hyperparameter configuration.

Using cross-validation ensures every data point is used for validation exactly once, reducing metric variance.

Illustration: 5-Fold Cross-Validation Splits

Below is a schematic of 5-Fold Cross-Validation. The dataset is split into 5 folds. The validation fold (orange) rotates, while the training folds (teal) are used to fit the model parameters:



Grid Search Optimization: GridSearchCV

To find the best combination of multiple hyperparameters, we perform a **Grid Search**:

- We define a grid of values for each hyperparameter:

$$\text{hidden\_layer\_sizes} \in \{\mathbf{H}_1 = (32, 16),\ \mathbf{H}_2 = (64, 32)\}$$
$$\text{activation} \in \{f_1 = \text{'relu'},\ f_2 = \text{'tanh'}\}$$
$$\text{alpha} \in \{\alpha_1 = 0.01,\ \alpha_2 = 0.1\}$$

- The search space is the Cartesian product of these sets, yielding $2 \times 2 \times 2 = 8$ configurations.
- For each configuration, we run 5-Fold Cross-Validation. This requires training:

$$8 \text{ configurations} \times 5 \text{ folds} = 40 \text{ training runs}$$

- The configuration with the highest averaged validation score (e.g., Macro F1) is selected as the best estimator and retrained on the entire training set.

Module 6: Model Diagnostics & Interpretability

Multiclass Diagnostics: Confusion Matrix

A **Confusion Matrix** is a diagnostic contingency table that shows where classification errors occur:

- **Rows:** Represent the actual true classes.
- **Columns:** Represent the model's predicted classes.
- **Diagonal Elements:** Represent correct predictions (True Positives).
- **Off-Diagonal Elements:** Highlight systematic errors (e.g., if Class 1: CC-SN is frequently misclassified as Class 2: Ia-SN).

	Predicted Label		
True Label	Background	CC-SN	Ia-S
Background	**40**	3	2
CC-SN	2	**6**	1
Ia-SN	1	1	**4**

In this example table, the model correctly predicted 40 background galaxies, 6 CC-SNe hosts, and 4 Ia-SNe hosts.

Multiclass Diagnostics: One-vs-Rest ROC & AUC

The Receiver Operating Characteristic (ROC) curve evaluates a classifier's sensitivity across different probability thresholds. For multiclass problems, we use the **One-vs-Rest (OvR)** method:

- We treat class c as the positive class, and group all other classes together as the negative class. We plot:

  $$\text{True Positive Rate (TPR)} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad \text{vs.} \quad \text{False Positive Rate (FPR)} = \frac{\text{FP}}{\text{FP} +$$

- We repeat this for each of the C classes, generating C separate ROC curves.
- **Area Under the Curve (AUC):** Integrates the area under the ROC curve:
  - AUC $= 0.5$ represents a random guess classifier.
  - AUC $= 1.0$ represents a perfect classifier.

Model Interpretability: The Black Box Challenge

In physics and astronomy, we do not just care about making accurate predictions; we must understand **why** the model made a prediction (what physical features it prioritized).

- In linear models (like linear regression), we can interpret weights directly:

$$y = w_1 x_1 + w_2 x_2 \implies w_i \text{ is the direct sensitivity.}$$

- In an MLP, features are passed through multiple layers of matrix multiplications and non-linear activations:

$$\mathbf{z}^{(L)} = \mathbf{W}^{(3)} f\left(\mathbf{W}^{(2)} f\left(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}\right) + \mathbf{b}^{(2)}\right) + \mathbf{b}^{(3)}$$

- Because the inputs are blended across hidden layers, we cannot interpret individual weights. The model acts as a **black box**.
- We resolve this using **Permutation Feature Importance**.

Permutation Feature Importance Theory

Let M be a trained classifier, $D = (\mathbf{X}, \mathbf{y})$ be the validation dataset, and s be the baseline score (e.g., Macro F1-score).

Breiman's Algorithm (2001)
For each physical feature j (e.g., Star Formation Rate):

1. **Shuffle Column:** Randomly shuffle the values of column j across the dataset. This breaks the physical correlation between feature j and the target label y while preserving the feature's distribution.
2. **Evaluate:** Calculate the model performance $s^{(j)}$ on this perturbed dataset.
3. **Compute Drop:** Define importance as the drop in performance:

$$I(j) = s - s^{(j)}$$

- If a feature is **critical**, shuffling it will degrade model performance, yielding a large $I(j) > 0$.
- If a feature is **irrelevant**, shuffling it has little effect, yielding $I(j) \approx 0$.

Summary & Best Practices

When building classifiers for physical datasets, follow these rules:

The Machine Learning Checklist

1. **Scale features safely:** Center and scale inputs using Standardization. Only fit the scaler on the training set to prevent data leakage.
2. **Split before oversampling:** Oversample training data only. Oversampling the validation/test set introduces leakage and yields artificially perfect metrics.
3. **Stratify splits:** Always stratify train-test splits for imbalanced datasets.
4. **Evaluate with Macro F1:** Do not rely on accuracy for imbalanced datasets. Use Macro F1 to weight rare classes equally.
5. **Interpret black-box models:** Use Permutation Feature Importance to verify that the network learns physically meaningful correlations.